NVX Stack Builder

คู่มือผู้ใช้ · User Guide

v0.5.1 · generated 2026-10-11

เริ่มต้นใช้งาน · Getting started

หน้านี้พาคุณจาก "ยังไม่รู้จักแอป" ไปจนได้สคริปต์ตั้งค่าโปรเจกต์ที่รันได้จริงภายในไม่กี่นาที พร้อมอธิบายว่าทำไมแต่ละขั้นจึงออกแบบมาแบบนี้

NVX Stack Builder คืออะไร

เวลาเริ่มโปรเจกต์ใหม่ นักพัฒนามักต้องค้นคำสั่งจากหลายแหล่ง เช่น ติดตั้ง Node ผ่าน nvm, สร้างแอปด้วย `create-next-app`, ติดตั้ง dependency, เพิ่ม ESLint/Vitest/Docker แล้วยังต้องจำว่าคำสั่งไหนต้องรันก่อนหลัง และบน Windows ต้องเขียนต่างกันอย่างไร NVX Stack Builder รวบรวมสิ่งเหล่านี้เป็น **Stack Template** คือชุดขั้นตอนที่เรียงลำดับแล้ว ตัดคำสั่งซ้ำแล้ว แต่ละขั้นมีคำอธิบายไทย/อังกฤษ คำสั่งแบบ POSIX และ PowerShell หมายเหตุตามระบบปฏิบัติการ ผลลัพธ์ที่คาดหวัง และคำสั่งตรวจสอบ (verify)

แนวคิดสำคัญคือ **แอปไม่รันคำสั่งแทนคุณ** ทุกอย่างเป็นข้อความที่คุณอ่าน ตรวจ แล้วรันเองบนเครื่อง เหตุผลคือคำสั่งตั้งค่าโปรเจกต์แตะไฟล์ระบบและติดตั้งซอฟต์แวร์ การให้มนุษย์ตรวจก่อนเสมอปลอดภัยกว่าและโปร่งใสกว่า (ดู Safety model)

สามเส้นทางเริ่มต้น

1. **เลือกเทมเพลตเอง** ไปที่หน้า `/templates` ค้นหาด้วยคำ เช่น `python`, `game`, `react` หรือกรองตามหมวด แล้วกดเปิดดูขั้นตอน เหมาะเมื่อรู้อยู่แล้วว่าต้องการ stack แบบไหน ดู templates.md
2. **ให้ AI agent วางแผน** ไปที่ `/builder` แท็บ "Ask the AI agent" แล้วพิมพ์อธิบายแอปเป็นภาษาไทยหรืออังกฤษ เช่น "แดชบอร์ดยอดขายพร้อม CI และเทสต์" agent จะเลือกเทมเพลตที่ใกล้ที่สุด เลือก add-on และอาจเพิ่มขั้นตอนพิเศษ ถ้าเซิร์ฟเวอร์ไม่มี API key ก็ยังใช้ได้ผ่าน offline planner ดู ai-agent.md
3. **เปิดลิงก์ที่มีคนแชร์มา** ลิงก์รูปแบบ `/builder#s=...` จะพาไปยังขั้น Review พร้อมสแต็กเดิมทุกประการ ดู sharing.md

ลองครั้งแรก (แนะนำ)

1. เปิดหน้าแรก กด **Ask the AI agent** หรือ **Browse templates**
2. ในตัวสร้าง (builder) ขั้น **Start** เลือก "Pick a template" แล้วเลือก *Next.js Analytics Dashboard*
3. ขั้น **Configure** ตั้งชื่อโปรเจกต์ เลือกเวอร์ชัน Node (LTS/24/22/20) ตัวจัดการแพ็กเกจ (npm/pnpm/yarn) และ add-on เช่น Testing หรือ Docker สังเกตว่าตัวสรุปสแต็ก (current stack) ด้านบนเปลี่ยนตาม
4. ขั้น **Review & reorder** อ่านแต่ละขั้น ลากเพื่อสลับลำดับ ลบขั้นที่ไม่ต้องการ หรือเพิ่มขั้นตอนของคุณเอง
5. ขั้น **Export** คัดลอกคำสั่งทั้งหมด หรือดาวน์โหลด `setup.sh` (macOS/Linux/WSL) / `setup.ps1` (Windows) / starter `.zip`
6. เปิดสคริปต์ อ่านทุกบรรทัด แล้วรัน `bash setup.sh` ในโฟลเดอร์ว่าง ขั้นที่เป็น dev server หรือ publish จะถูกคอมเมนต์ไว้โดยตั้งใจ ให้รันเองในเทอร์มินัลแยก

สิ่งที่ควรมีบนเครื่อง

- **Node.js** ผ่าน nvm (macOS/Linux) หรือ nvm-windows สำหรับเทมเพลตฝั่ง JavaScript ขั้นติดตั้ง nvm เป็นขั้น `requires: nvm` สคริปต์จะข้ามพร้อมคำเตือนถ้าไม่พบ nvm
- **Python 3.11–3.13** หรือ **uv** สำหรับเทมเพลตฝั่ง Python
- Git และเทอร์มินัล (bash/zsh หรือ PowerShell)

คำแนะนำ

- เริ่มจากค่าเริ่มต้นของเทมเพลตก่อน แล้วค่อยเพิ่ม add-on ทีละตัว เพื่อให้รู้ว่าแต่ละตัวเพิ่มขั้นตอนอะไร
- อ่านคอลัมน์ "Expected result" และรัน "Verify" หลังแต่ละขั้น จะจับปัญหาได้เร็วกว่ารันรวดเดียวแล้วค่อยหาว่าพังตรงไหน
- เก็บลิงก์แชร์ไว้ในโน้ตของโปรเจกต์ เพื่อสร้างสแต็กเดิมซ้ำได้ภายหลัง

อ้างอิง

- โค้ดหน้าแรก `src/app/page.tsx`, ตัวสร้าง `src/components/builder.tsx`
- ภาพหน้าจอเวอร์ชันล่าสุด: `screenshots/v0.5/`

การใช้ Stack Template · Using templates

Stack Template คือ "สูตร" ตั้งค่าโปรเจกต์ที่คัดสรรแล้ว แต่ละเทมเพลตถูกเขียนเป็นไฟล์ข้อมูล TypeScript ใน `src/data/templates/` (ไม่ใช่โค้ดที่ซ่อนอยู่ในคอมโพเนนต์) จึงตรวจทาน ทดสอบ และเพิ่มใหม่ได้ง่าย แนวคิดเต็มอยู่ที่ concepts/stack-template.md

เทมเพลตทั้ง 10 แบบ (ณ v0.5)

id	ชื่อ	runtime	add-on ที่รองรับ	ค่าเริ่มต้น
`nextjs-dashboard`	Next.js Analytics Dashboard	Node	typescript, tailwind, eslint, testing, docker, ci	typescript, tailwind, eslint
`vite-react-game`	Vite + React Canvas Game	Node	typescript, tailwind, eslint, testing, docker, ci	typescript
`express-api`	Express REST API	Node	testing, docker, ci	—
`fastapi-ai`	FastAPI + Python AI App	Python	eslint (Ruff), testing, docker, ci	—
`streamlit-dashboard`	Streamlit Data Dashboard	Python	eslint (Ruff), testing, docker, ci	—
`node-cli-npx`	Node CLI Tool (run via npx)	Node	testing, ci	—
`pypi-package`	Python Package for PyPI	Python	eslint (Ruff), testing, ci	testing
`collab-workspace`	Realtime Collaborative Workspace	Node	typescript, tailwind, eslint, testing, docker, ci	typescript, tailwind, eslint
`react-admin-tool`	Internal Tool (React Admin)	Node	typescript, eslint, testing, docker, ci	typescript
`nextjs-ai-chat`	Next.js AI Chat App	Node	typescript, tailwind, eslint, testing, docker, ci	typescript, tailwind, eslint

ตารางนี้ตรวจกับโค้ดอัตโนมัติด้วย `npm run docs:check` (ทุก template id ต้องปรากฏในเอกสาร) ถ้าเพิ่มเทมเพลตใหม่แล้วไม่อัปเดตเอกสาร CI จะล้ม

หน้า catalog (`/templates`)

- **ช่องค้นหา** ค้นจากชื่อ คำอธิบาย และแท็ก เช่น `python` , `game` , `react`
- **ชิปหมวดหมู่** (เช่น Dashboard, Game, API, AI app, Data, CLI, Package, Collaborative, Internal tool ตาม `CATEGORY_LABELS` ใน `src/lib/i18n.ts`) ใช้กรองผล ถ้าไม่พบอะไรจะมีปุ่ม "Clear filters"
- การ์ดแต่ละใบแสดงไทล์โลโก้เครื่องมือหลัก (เช่น Next.js, FastAPI) หมวด runtime แท็ก และปุ่ม **View steps** / **Open in builder**

หน้ารายละเอียดเทมเพลต (`/templates/<id>`)

แสดงทุกขั้นตอนตามลำดับที่ composer สร้างจากค่าเริ่มต้น (ค่าเริ่มต้น: npm, Node LTS, Python 3.12 และ add-on เริ่มต้นของเทมเพลต) แต่ละขั้นมี:

- **ชื่อและคำอธิบาย** ว่าทำไมต้องทำขั้นนี้ ไม่ใช่แค่ทำอะไร
- **คำสั่ง** สลับดูแบบ macOS/Linux หรือ Windows PowerShell ได้ (ปุ่มด้านบนรายการ)
- **OS notes** ข้อควรระวังเฉพาะระบบ เช่น nvm-windows
- **Expected result** และ **Verify** คำสั่งพิสูจน์ว่าขั้นนั้นสำเร็จ
- ป้าย **run manually / dev server** สำหรับขั้นที่ไม่ควรรันอัตโนมัติ

แถบด้านข้างมี file tree โดยประมาณ ขั้นตอนถัดไป (next steps) และลิงก์เอกสารทางการ id ที่ไม่มีอยู่จะได้ HTTP 404 จริง (ตั้งแต่ v0.3.0, commit `b88a5c5`)

คำแนะนำในการเลือก

- ต้องการ UI + API ใน repo เดียว → `nextjs-dashboard` หรือ `nextjs-ai-chat`
- API ล้วน → `express-api` (Node) หรือ `fastapi-ai` (Python)
- งานข้อมูลเร็ว ๆ → `streamlit-dashboard`
- เผยแพร่แพ็กเกจ → `node-cli-npx` (npm) หรือ `pypi-package` (PyPI) ขั้น publish จะถูกคอมเมนต์ไว้เสมอ
- ปุ่ม **Open in builder** เปิด `/builder#t=<id>` ที่ขั้น Configure เพื่อปรับต่อ

อ้างอิง

`src/components/catalog.tsx` , `src/components/template-detail.tsx` , `src/app/templates/[id]/page.tsx` , schema: reference/template-schema.md

ตัวสร้างสแต็ก 4 ขั้น · The builder

ตัวสร้าง (`/builder`) คือหัวใจของแอป มันรับ "การตั้งค่า" (`BuilderConfig`) กับ "การแก้ไขของผู้ใช้" (`BuilderEdits`) แล้วคำนวณลำดับคำสั่งสุดท้ายแบบเรียลไทม์ หน้านี้อธิบายทั้งวิธีใช้และเหตุผลของแต่ละขั้น ตัวสร้างถูกออกแบบใหม่เป็น wizard 4 ขั้นใน v0.2.0 (commit `853feac`) เพื่อลดภาระการคิดเมื่อเทียบกับหน้าเดียวยาว ๆ ใน v0.1.0

ภาพรวมขั้นตอน (Stepper)

ขั้น	ชื่อ	ทำอะไร	ทำไมต้องแยกขั้น
1	**Start**	ถาม AI หรือเลือกเทมเพลต	ตัดสินใจเรื่องใหญ่ที่สุด (ประเภทแอป) ก่อน
2	**Configure**	ชื่อโปรเจกต์, เวอร์ชัน Node/Python, npm/pnpm/yarn, pip/uv, add-on	ตัวเลือกพวกนี้เปลี่ยนคำสั่งทุกขั้น จึงต้องตั้งก่อนจัดลำดับ
3	**Review & reorder**	อ่าน ลาก ลบ กู้คืน เพิ่มขั้นตอนเอง	ปรับแต่งบนลำดับที่ถูกต้องอยู่แล้ว แทนการเริ่มจากศูนย์
4	**Export**	คัดลอก แชร์ ดาวน์โหลด	ส่งมอบผลลัพธ์ในรูปแบบที่เหมาะกับแต่ละ OS

แถบสรุป **Current stack** ด้านบนแสดงเทมเพลต ชื่อโปรเจกต์ runtime ตัวจัดการแพ็กเกจ add-on และจำนวนขั้นตลอดเวลา พร้อมปุ่ม Copy ปุ่ม Back/Next ด้านล่างเลื่อนขั้น และโฟกัสคีย์บอร์ดจะย้ายไปที่หัวข้อขั้นใหม่เพื่อการเข้าถึง (accessibility)

ขั้น 1: Start

- แท็บ **Ask the AI agent** อธิบายแอปแล้วกด *Plan my stack* หรือ *Use offline planner* (ดู ai-agent.md) เมื่อได้แผนแล้วกด **Load into builder** เทมเพลต add-on และขั้นตอนพิเศษจาก agent จะถูกโหลดเข้า
- แท็บ **Pick a template** เลือกการ์ดเทมเพลตจากรายการ เหมือนหน้า catalog

ขั้น 2: Configure

- **Project name** ถูกทำให้ปลอดภัยด้วย `sanitizeProjectName` (ตัวพิมพ์เล็ก a-z 0-9 . _ - ยาวไม่เกิน 50) เพราะชื่อนี้กลายเป็นชื่อโฟลเดอร์และชื่อแพ็กเกจ npm/PyPI
- **Node.js version (nvm)**: LTS, 24, 22, 20 และ **JS package manager**: npm / pnpm / yarn เลือก pnpm หรือ yarn จะเพิ่มขั้นเปิดใช้ตัวจัดการนั้นอัตโนมัติ
- **Python version**: 3.13, 3.12, 3.11 และ **Python package manager**: pip / uv เลือก uv จะเพิ่มขั้นติดตั้ง uv และติดตั้ง Python ผ่าน uv
- **Add-ons**: TypeScript, Tailwind CSS, Linting (ESLint / Ruff), Testing (Vitest / pytest), Docker, GitHub Actions CI add-on ที่เทมเพลตไม่รองรับจะถูกปิดและมีข้อความ "not available for this template"

ทุกตัวเลือกถูกแปลงเป็น placeholder เช่น `{{install}}`, `{{add}}`, `{{nextFlags}}` ใน composer จึงไม่ต้องเขียนเทมเพลตซ้ำสำหรับแต่ละตัวจัดการแพ็กเกจ (ดู composition.md)

ขั้น 3: Review & reorder

- **ลากที่จับ** เพื่อสลับลำดับ หรือใช้คีย์บอร์ด: โฟกัสที่จับ กด Space เพื่อยก กด ↑/↓ เพื่อย้าย กด Space อีกครั้งเพื่อวาง มีปุ่มเลื่อนขึ้น/ลงสำรองด้วย (การย้ายด้วยคีย์บอร์ดถูกแก้ให้ขยับทีละหนึ่งช่องเสมอใน commit `ae35a20`)
- **ลบ** ขั้นตอนแล้วกู้คืนได้จากส่วน "Removed steps" มี toast ยืนยันทุกครั้ง
- **Add a custom step** ใส่ชื่อและคำสั่ง (บรรทัดละคำสั่ง) คำสั่งผ่าน `sanitizeStep` เสมอ คำสั่งอันตราย เช่น `rm -rf /` ถูกบล็อก ส่วนคำสั่งเสี่ยง เช่น `curl ... | sh` หรือ `sudo` ถูกทำเครื่องหมายเป็น manual
- **Reset edits** ย้อนกลับเป็นลำดับเริ่มต้นของเทมเพลต

ขั้นตอนที่ผู้ใช้หรือ AI เพิ่มจะถูกแทรก **ก่อน** ขั้นแรกที่เป็น long-running/publish/manual เพื่อให้ dev server อยู่ท้ายสุดเสมอ (ตรรกะใน `applyEdits`)

ขั้น 4: Export

- สลับ **Shell**: POSIX (bash/zsh) หรือ Windows PowerShell แล้วดูคำสั่งทั้งหมดในหน้าต่างโค้ด
- **Copy all commands**, **Copy share link**, ดาวน์โหลด `setup.sh`, `setup.ps1`, `README.md`, **Starter .zip** (ดู downloads.md)

การจำสถานะ

ทุกการเปลี่ยนแปลงถูกเข้ารหัสลง URL hash (`/builder#s=...`) และ `localStorage` คีย์ `nvx-builder-state` ทันที รีเฟรชหน้าแล้วได้สแต็กเดิม และ URL ในแถบที่อยู่คือ share link เสมอ

อ้างอิง

`src/components/builder.tsx`, `src/components/sortable-steps.tsx`, `src/lib/composer.ts`, `src/lib/placeholders.ts`

AI agent planner

AI agent ช่วยแปลง "คำอธิบายแอปเป็นภาษาคน" ให้เป็นแผนสแต็ก (`AgentPlan`) ที่โหลดเข้าตัวสร้างได้ทันที หน้านี้อธิบายสามโหมดการทำงาน เหตุผลของการออกแบบ และสิ่งที่ควรรู้เรื่องความเป็นส่วนตัวและค่าใช้จ่าย

หลักการออกแบบ

AI **ไม่ได้เขียนคำสั่งตั้งค่าหลักเอง** มันเลือกเทมเพลตที่ใกล้ที่สุดจาก catalog (ส่ง `templateId` เป็น enum), เลือก add-on, ตัวจัดการแพ็กเกจ แล้ว *อาจ* เพิ่ม `extraSteps` (0–6 ขั้น) สำหรับงานนอกเทมเพลต เช่น ติดตั้ง database client คำสั่งหลักยังมาจาก composer ซึ่งผ่านการทดสอบแล้ว เหตุผล: ลด hallucination และทำให้ผลลัพธ์ทำซ้ำได้ ส่วนที่มาจาก AI ทั้งหมดผ่าน zod validation และ sanitizer ก่อนถึงหน้าจอ ขั้นตอนที่ซ้ำกับคำสั่งของเทมเพลตจะถูกตัดออก

สามโหมด

1. **ใช้ key ของเซิร์ฟเวอร์ (Server AI)** ถ้าผู้ดูแลตั้ง `OPENAI_API_KEY` ไว้ ป้ายบนแผงจะแสดง "Server AI: key configured · <model>" (ข้อมูลจาก `GET /api/plan`) คำขอเรียก OpenAI Responses API แบบ Structured Outputs ด้วยโมเดลจาก `OPENAI_MODEL` (ค่าเริ่มต้น `gpt-6.1-sol`) โหมดนี้มี rate limit เข้มกว่า: 5 ครั้งต่อนาทีต่อ IP เพื่อปกป้องบิลของผู้ดูแล
2. **ใช้ key ของตัวเอง (own key)** ติ๊ก "Use my own OpenAI API key" แล้ววาง key key จะถูกส่งไปใน header `x-openai-key` ของคำขอนั้นเท่านั้น เซิร์ฟเวอร์ไม่เก็บ ไม่ log และ redact ออกจากข้อความ error โหมดนี้ไม่ติด AI limiter (ติดเฉพาะ limiter ทั่วไป 20 ครั้งต่อนาที) เพราะใช้เงินของผู้ใช้เอง
3. **Offline planner (rule-based fallback)** กด "Use offline planner" หรือจะถูกใช้อัตโนมัติเมื่อไม่มี key, OpenAI error, timeout หรือผลลัพธ์ไม่ผ่าน validation ตัววางแผนนี้ให้คะแนนเทมเพลตจาก `keywords` (ไทย+อังกฤษ) และคำใบ้ runtime เช่น "python", "fastapi" จับ add-on จากคำ เช่น "docker", "ทดสอบ", "github actions" และดึงชื่อโปรเจกต์จากวลี "called ..." หรือ "ชื่อ ..." ผลลัพธ์ระบุ `source: "fallback"` และ `fallbackReason`

วิธีเขียน prompt ให้ได้ผลดี

- บอก **ประเภทแอป** + **runtime ที่ต้องการ** + **add-on** เช่น "FastAPI service ที่เรียก OpenAI พร้อม pytest และ Docker ใช้ uv"
- ระบุชื่อโปรเจกต์ด้วยวลี `named my-app` หรือ `ชื่อ my-app`
- ไม่ต้องสั่งให้ AI ทำสิ่งนอกขอบเขต คำสั่งในข้อความผู้ใช้ที่ขัดกับกฎระบบจะถูกเพิกเฉย (prompt injection guard ใน system instructions)
- prompt ยาวได้ไม่เกิน 2,000 ตัวอักษรหลัง sanitize และต้องยาวอย่างน้อย 3 ตัวอักษร

อ่านผลลัพธ์

แผงผลลัพธ์แสดง: ป้าย *AI plan* หรือ *Rule-based plan (offline)*, เหตุผล fallback, เทมเพลตที่เลือก, คำสั่งตามลำดับ, ขั้นตอนพิเศษจาก agent, ไฟล์เริ่มต้นสำคัญ และ **Safety notes** เช่น "Removed 1 unsafe or invalid command(s)" หรือขั้นที่ต้องตรวจเอง กด **Load into builder** เพื่อโหลดเข้าตัวสร้าง (ไฟล์เริ่มต้นจาก AI จะกลายเป็นขั้น `custom-ai-files`)

ข้อผิดพลาดที่พบบ่อย

- **429 Too many requests** รอประมาณ 60 วินาที (header `Retry-After: 60`) หรือใช้ offline planner
- แผงขึ้น error state: กด *Try again* หรือ *Use offline planner*
- ได้ fallback ทั้งที่มี key: ดู `fallbackReason` เช่น `OpenAI 401` (key ผิด) หรือ `AI request timed out`

อ้างอิง

`src/components/agent-panel.tsx`, `src/app/api/plan/route.ts`, `src/lib/ai/openai.ts`, `src/lib/ai/fallback.ts`, `src/lib/ai/plan.ts`, API: reference/api-plan.md, สถาปัตยกรรม: architecture/ai-planner.md

ลิงก์แชร์ · Sharing links

ลิงก์แชร์ทำให้คุณส่ง "สแต็กที่ปรับแต่งแล้ว" ให้เพื่อนร่วมทีม หรือเก็บไว้สร้างซ้ำในอนาคตได้ โดยไม่ต้องมีบัญชีผู้ใช้หรือฐานข้อมูล

รูปแบบลิงก์

รูปแบบ	ตัวอย่าง	เปิดแล้วไปที่
Share link	`/builder#s=eyJ2Ijox...`	ขั้น **Review & reorder** พร้อม config และการแก้ไขทั้งหมด และมี toast "Loaded a shared stack from the link."
Template link	`/builder#t=fastapi-ai`	ขั้น **Configure** ของเทมเพลตนั้นด้วยค่าเริ่มต้น (ปุ่ม Open in builder ใช้รูปแบบนี้)
Pick tab	`/builder#pick`	ขั้น Start โดยเปิดแท็บ Pick a template

ถ้าไม่มี hash ตัวสร้างจะโหลดสถานะล่าสุดจาก `localStorage` (`nvx-builder-state`) หรือค่าเริ่มต้น (`nextjs-dashboard`)

สิ่งที่อยู่ในลิงก์ และทำไมใช้ URL hash

token หลัง `#s=` คือ JSON ที่เข้ารหัสแบบ base64url มีฟิลด์: `v` (เวอร์ชันรูปแบบ = 1), `c` (config), `o` (ลำดับ id ที่ผู้ใช้จัด), `r` (id ที่ลบ), `x` (ขั้นตอนที่เพิ่มเอง) ส่วนที่ว่างจะไม่ถูกใส่เพื่อให้ลิงก์สั้น

เหตุผลที่ใช้ **fragment (`#`)** ไม่ใช่ query string: เบราว์เซอร์ไม่ส่ง fragment ไปยังเซิร์ฟเวอร์ จึงไม่มีข้อมูลสแต็กไปโผล่ใน log ของเซิร์ฟเวอร์หรือ CDN และไม่ต้องมีฐานข้อมูลเก็บลิงก์ ข้อเสียคือลิงก์อาจยาวถ้ามีขั้นตอนเพิ่มเองเยอะ (จำกัด token ไม่เกิน 60,000 ตัวอักษร)

ความปลอดภัยของลิงก์ที่ได้รับ

ลิงก์แชร์ถือเป็น **ข้อมูลไม่น่าเชื่อถือ (untrusted)** เพราะใครก็สร้างได้ ตอนเปิด `decodeShare` จะ:

- ตรวจรูปแบบด้วย zod schema (จำกัดจำนวน: order/removed ≤ 100 รายการ, custom ≤ 20 ขั้น)
- ทำ config ให้ถูกต้องด้วย `normalizeConfig` (ค่าที่ไม่รู้จักกลับเป็นค่าเริ่มต้น, ชื่อโปรเจกต์ถูก sanitize)
- ส่งทุกขั้นตอนที่เพิ่มเองผ่าน `sanitizeStep` คำสั่งทำลายระบบถูกตัดทิ้ง คำสั่งเสี่ยงกลายเป็น manual (คอมเมนต์ไว้ในสคริปต์)
- token เสียหรือไม่ถูกต้อง → คืน `null` และตัวสร้างใช้ค่าเริ่มต้นแทน ไม่ crash

แม้อย่างนั้น ให้ **อ่านสคริปต์ก่อนรันทุกครั้ง** โดยเฉพาะลิงก์จากคนที่ไม่รู้จัก

วิธีแชร์

1. ไปขั้น Export แล้วกด **Copy share link** (หรือคัดลอก URL จากแถบที่อยู่ ซึ่งอัปเดตตลอด)

2. ส่งลิงก์ ผู้รับเปิดแล้วจะได้สแต็กเดียวกันทุกประการ
3. ไฟล์ `README.md` และ `nvx-stack.json` ในไฟล์ zip ก็มีลิงก์นี้ด้วย

ความเข้ากันได้ข้ามเวอร์ชัน

รูปแบบ `v: 1` ไม่เปลี่ยนตั้งแต่ v0.1.0 (commit `a452dc5`) ถ้า id ของขั้นตอนในเทมเพลตเปลี่ยนภายหลัง ลำดับที่อ้าง id เก่าจะถูกข้าม และขั้นใหม่ถูกแทรกต่อจากขั้นก่อนหน้าตามธรรมชาติ (`applyEdits`) ถ้าต้องเปลี่ยนรูปแบบในอนาคต ให้เพิ่ม `v: 2` และคงตัวถอดรหัส `v: 1` ไว้

อ้างอิง

`src/lib/share.ts`, `src/lib/sanitize.ts`, `src/components/builder.tsx`, สถาปัตยกรรม: scripts-and-share.md

การดาวน์โหลด · Downloads (setup.sh / setup.ps1 / README / zip)

ขั้น **Export** ของตัวสร้างมีไฟล์ให้ดาวน์โหลด 4 แบบ ทั้งหมดสร้างในเบราว์เซอร์ของคุณ (client-side) จากขั้นตอนที่แสดงอยู่ ไม่มีการส่งไปเซิร์ฟเวอร์

ไฟล์แต่ละแบบ

ไฟล์	สำหรับ	สร้างโดย
`setup.sh`	macOS / Linux / WSL (bash)	`toBash()`
`setup.ps1`	Windows PowerShell	`toPowerShell()` (บรรทัดลงท้าย CRLF)
`README.md`	เอกสารโปรเจกต์ไทยหรืออังกฤษตามภาษาที่เลือก	`toReadme()`
`<project>-starter.zip`	ทั้งหมดข้างบน + `nvx-stack.json` + โฟลเดอร์ `starter-files/`	`buildStarterZip()`

โครงสร้างของ setup.sh และเหตุผล

1. **โหลด nvm ก่อน** (`. "$NVM_DIR/nvm.sh"`) เพราะ nvm เป็น shell function ไม่ใช่ไฟล์โปรแกรม สคริปต์ที่ไม่ได้ source จะหา `nvm` ไม่เจอ
2. `set -eo pipefail` หยุดทันทีเมื่อคำสั่งใดล้มเหลว เพื่อไม่ให้ขั้นต่อไปทำงานบนสถานะที่พัง
3. ฟังก์ชัน `step` พิมพ์หัวข้อ `==> 3/7 ...` และ `verify` รันคำสั่งตรวจสอบ ถ้าไม่ผ่านจะเตือน `[warn]` แต่ไม่หยุดสคริปต์
4. ไฟล์เริ่มต้นเขียนด้วย heredoc แบบ `<<'NVX_EOF'` (มี quote) จึงไม่มีการแทนค่าตัวแปร sanitizer ปฏิเสธไฟล์ที่มีบรรทัด `NVX_EOF` เพื่อกันการหลุดออกจาก heredoc
5. ขั้นที่มี `requires` (เช่น nvm) ถูกห่อด้วย `if command -v nvm ...` ถ้าไม่พบจะข้ามพร้อมคำเตือน
6. ขั้นชนิด **longRunning** (dev server), **publish** (npm publish, twine upload) และ **manual** ถูก **คอมเมนต์ไว้** พร้อมหมายเหตุ เพราะ dev server จะค้างสคริปต์ไว้ และการ publish เป็นการกระทำสาธารณะที่ย้อนกลับไม่ได้

`setup.ps1` ทำแบบเดียวกันด้วย `$ErrorActionPreference = "Stop"`, ฟังก์ชัน `Step`, `Assert-Ok` (ตรวจ `$LASTEXITCODE` หลังทุกคำสั่ง), `Write-NvxFile` (เขียนไฟล์ด้วย here-string `@' ... '@`) และ `Verify`

วิธีรันอย่างปลอดภัย

```
mkdir ~/projects && cd ~/projects
less setup.sh          # อ่านทุกบรรทัดก่อน
bash setup.sh
```

```
Get-Content .\setup.ps1 | more
powershell -ExecutionPolicy Bypass -File .\setup.ps1
```

`-ExecutionPolicy Bypass` ใช้เฉพาะการรันครั้งนี้ ไม่เปลี่ยน policy ของเครื่อง

ภายใน starter zip

```
<project>-starter/
  setup.sh            (สิทธิ์ 755)
  setup.ps1
  README.md           (มี share link)
  nvx-stack.json      (generator, shareUrl, config, steps[] แบบอ่านด้วยเครื่องได้)
  starter-files/...   (ทุกไฟล์ที่ขั้นตอนต่าง ๆ จะเขียน ณ path เดิม)
```

`nvx-stack.json` มีประโยชน์สำหรับเครื่องมืออัตโนมัติหรือ AI agent ที่ต้องการอ่านลำดับขั้นตอนโดยไม่ต้อง parse shell script

คำแนะนำ

- รันในโฟลเดอร์ว่างเสมอ ขั้นแรกของเทมเพลตส่วนใหญ่สร้างโฟลเดอร์ `{{name}}` แล้ว `cd` เข้าไป
- หลังสคริปต์จบ ให้รันขั้นที่คอมเมนต์ไว้ (เช่น `npm run dev`) ในเทอร์มินัลแยก
- ผู้ร่วมพัฒนาสร้างสคริปต์จาก CLI ได้ด้วย `npm run export -- <templateId>` (ดู npm-scripts.md)

อ้างอิง

`src/lib/scripts.ts`, `src/lib/download.ts`, `scripts/export-template.ts`

ภาษา ธีม และการนำทาง · Language, theme & navigation

แอปรองรับภาษาไทยและอังกฤษเต็มรูปแบบ มีธีมดำ (ค่าเริ่มต้น) และธีมสว่าง และมี command palette สำหรับผู้ที่ชอบใช้คีย์บอร์ด หน้านี้อธิบายวิธีใช้และเหตุผลของการออกแบบ

ภาษา (TH / EN)

- ปุ่ม **ภาษาไทย / English** (ไอคอน Languages) บนแถบด้านบน สลับภาษาของทั้งแอปทันที ค่าถูกจำใน `localStorage` คีย์ `nvx-lang` ถ้ายังไม่เคยเลือก แอปจะใช้ภาษาไทยเมื่อภาษาของเบราว์เซอร์ (`navigator.language`) ขึ้นต้นด้วย `th` และใช้อังกฤษในกรณีอื่น
- ข้อความ UI ทั้งหมดอยู่ใน `src/lib/i18n.ts` ส่วนเนื้อหาของเทมเพลตเป็นชนิด `L10n` (`{ en, th }`) ที่ต้องมีทั้งสองภาษาเสมอ
- ภาษาที่เลือกมีผลกับ: คำอธิบายขั้นตอน, หัวข้อใน `setup.sh` / `setup.ps1`, `README.md` ที่ดาวน์โหลด และภาษาของคำตอบจาก AI agent (ส่ง `lang` ไปกับคำขอ)
- คำสั่งและโค้ดเป็นภาษาอังกฤษเสมอ เพราะต้องรันได้จริง
- ฟอนต์ไทยคือ **IBM Plex Sans Thai** (ตั้งแต่ v0.2.0) หัวข้อภาษาไทยใช้ letter-spacing ปกติและ line-height สูงขึ้น เพื่อไม่ให้สระและวรรณยุกต์ชนกัน

ธีม (Dark / Light)

- **ธีมดำเป็นค่าเริ่มต้นสำหรับทุกคน** ไม่ขึ้นกับการตั้งค่า OS (v0.1–v0.3 ใช้ `prefers-color-scheme` ของ OS; เปลี่ยนใน v0.4, commit `2210920`, ตามความต้องการของเจ้าของที่อยากได้ลุคเครื่องมือนักพัฒนา) ปุ่มพระอาทิตย์/พระจันทร์สลับเป็นธีมสว่าง ค่าถูกจำใน `localStorage` คีย์ `nvx-theme`
- เซิร์ฟเวอร์เรนเดอร์ `<html class="dark">` ไว้ก่อน และสคริปต์ pre-paint เล็ก ๆ ใน `layout.tsx` ถอดคลาสออกเฉพาะเมื่อผู้ใช้เลือก light จึงไม่มีการกะพริบสี (flash) ตอนโหลด
- ดีไซน์ v0.5 อ้างอิง developer.apple.com: พื้นดำ ตัวหนังสือใหญ่ ปุ่มทรงแคปซูล สีฟ้าใช้อย่างจำกัด และโลโก้เครื่องมือจริงเป็นไทล์ (ดู concepts/design-system.md)
- ผู้ใช้ที่ตั้ง **prefers-reduced-motion** จะได้แอนิเมชันเกือบเป็นศูนย์ และเคอร์เซอร์ในหน้าต่างโค้ดหยุดกะพริบ

Command palette

- กด **Ctrl + K** (Windows/Linux) หรือ **⌘ + K** (macOS) หรือคลิกช่อง "Search or jump to..."
- พิมพ์เพื่อค้นหา: หน้าต่าง ๆ (Home, Templates, Builder, Design system), เทมเพลตทุกตัว (เปิดหน้ารายละเอียดหรือเปิดในตัวสร้าง) และ action สลับธีม/ภาษา
- ↑↓ เลื่อน, Enter เปิด, Esc ปิด

การนำทางบนมือถือ

บนจอเล็ก เมนูจะยุบเป็นปุ่ม hamburger ที่เปิด sheet ด้านข้าง ทุกหน้าถูกทดสอบที่ความกว้าง 390px ว่าไม่มี horizontal scroll ทั้งภาษาไทยและอังกฤษ

อ้างอิง

`src/components/prefs.tsx`, `src/app/layout.tsx`, `src/components/header.tsx`, `src/components/command-palette.tsx`

แก้ปัญหาและคำถามที่พบบ่อย · Troubleshooting & FAQ

รวมปัญหาที่ผู้ใช้และผู้ดูแลเจอบ่อย พร้อมสาเหตุและวิธีแก้ ถ้าไม่พบคำตอบ ดูหน้า operations สำหรับฝั่งเซิร์ฟเวอร์

ฝั่งผู้ใช้

`nvm: command not found` หรือขั้นติดตั้ง Node ถูกข้าม nvm เป็น shell function ต้องติดตั้งก่อน (ดู OS notes ในขั้นแรก) แล้วเปิดเทอร์มินัลใหม่ สคริปต์จะพยายาม source `$NVM_DIR/nvm.sh` ให้อัตโนมัติ ถ้าไม่พบจะพิมพ์ `[warn] nvm not found — skipping` แทนการล้มทั้งสคริปต์ บน Windows ใช้ nvm-windows (คำสั่ง `nvm install lts`)

สคริปต์หยุดกลางทาง `setup.sh` ใช้ `set -eo pipefail` และ `setup.ps1` ใช้ `Assert-Ok` จึงหยุดที่คำสั่งแรกที่ล้มเหลว ดูหัวข้อ `==> n/total` ล่าสุดเพื่อรู้ว่าขั้นไหน แก้สาเหตุ (เช่น เน็ตหลุด, เวอร์ชัน Node ไม่ตรง) แล้วรันคำสั่งที่เหลือด้วยมือ หรือลบโฟลเดอร์โปรเจกต์แล้วรันใหม่

`[warn] verification failed` คำสั่ง verify ไม่ผ่าน แต่สคริปต์ยังทำต่อ ให้อ่าน "Expected result" ของขั้นนั้นแล้วตรวจด้วยมือ บางครั้งเป็นเพราะ PATH ยังไม่อัปเดตจนกว่าจะเปิดเทอร์มินัลใหม่

PowerShell ไม่ยอมรันสคริปต์ ใช้ `powershell -ExecutionPolicy Bypass -File .\setup.ps1` ซึ่งข้าม policy เฉพาะครั้งนี้

ทำไม `npm run dev` ไม่ถูกรัน ขั้น dev server เป็นชนิด `longRunning` ถูกคอมเมนต์ไว้โดยตั้งใจ เพราะจะค้างสคริปต์ ให้รันเองในเทอร์มินัลแยก เช่นเดียวกับขั้น publish

คำสั่งที่ฉันเพิ่มเองหายไป / มีข้อความ "That command was blocked" sanitizer บล็อกคำสั่งทำลายระบบ (เช่น `rm -rf /`, `mkfs`, `dd if=`, fork bomb, `shutdown`) และคำสั่งยาวเกิน 400 ตัวอักษร ส่วนคำสั่งเสี่ยง (`curl | sh`, `sudo`, `eval`, `npm publish`, `git push --force`) ยังอยู่แต่ถูกทำเป็น manual ดู safety-model.md

AI ตอบ "Too many requests" เกิน rate limit (20 POST/นาที/IP ทุกโหมด และ 5 ครั้ง/นาที/IP เมื่อใช้ key ของเซิร์ฟเวอร์) รอ 60 วินาที หรือใช้ offline planner หรือใช้ key ของตัวเอง

ได้แผน "Rule-based (offline)" ทั้งที่อยากได้ AI ดู "Fallback reason" ในแผง: `no OpenAI API key configured` (เซิร์ฟเวอร์ไม่มี key), `OpenAI 401` (key ผิด), `AI request timed out`, `invalid AI output`

ลิงก์แชร์เปิดแล้วได้ค่าเริ่มต้น token เสีย (ถูกตัดตอนคัดลอก) หรือเกิน 60,000 ตัวอักษร ให้คัดลอกใหม่ทั้งบรรทัด

FAQ

- **แอปรันคำสั่งบนเซิร์ฟเวอร์ไหม?** ไม่ ไม่มีโค้ดเส้นทางใดที่ execute คำสั่ง ทุกอย่างเป็นข้อความ
- **key ของฉันถูกเก็บไหม?** ไม่ ใช้เฉพาะคำขอเดียว ไม่ log และ redact ออกจาก error
- **ข้อมูลสแต็กถูกส่งไปไหน?** share link อยู่ใน URL fragment ซึ่งไม่ถูกส่งไปเซิร์ฟเวอร์ prompt ถูกส่งไปที่ `/api/plan` (และ OpenAI ถ้าใช้ AI โดยตั้ง `store: false`) และไม่ถูก log

- **รองรับ Windows ไหม?** รองรับผ่าน `setup.ps1` คำสั่งที่ไม่มี override เฉพาะ Windows จะถูกแปลงแบบ best-effort (`curl` → `curl.exe`, `python3` → `py -3.12`, `&&` → `;`)
- **เพิ่มเทมเพลตได้ไหม?** ได้ ดู add-template.md

อ้างอิง

`src/lib/sanitize.ts`, `src/lib/scripts.ts`, `src/lib/composer.ts`